

Projet Python — Groupe 3

Gestion des erreurs, classes, numpy et matplotlib

Enseignante : Rahma GARGOURI — 2026-2027

Présentation finale — Projet en groupe

Vous préparez une **présentation** (PowerPoint, PDF ou autre format) qui répond aux questions ci-dessous, accompagnée d'un **notebook Jupyter** contenant les solutions des exercices, à exécuter pendant la présentation.

Partie A — Gestion des erreurs et journalisation

Questions

1. Qu'est-ce qu'une exception ? Que se passe-t-il si elle n'est pas gérée ?
2. Pourquoi un `except:` sans type d'erreur est-il déconseillé ?
3. Quelle est la différence entre `else` et `finally` ?
4. Pourquoi utiliser `logging` plutôt que `print` ? Citez les niveaux de log et un usage de chacun.

Exercices

A.1 — Lire un montant. Écrire `lire_montant(texte)` qui convertit "12,5" ou "12.5" en 12.5, et renvoie `None` si la conversion est impossible, en écrivant un avertissement dans un fichier de log.

A.2 — Division sûre. Écrire `division_sure(a, b)` qui gère la division par zéro. Utiliser `else` ou `finally` et montrer quand ils s'exécutent.

Partie B — Classes : la pile

Notion nouvelle (non vue en séance). Une **pile** est une structure de données qui fonctionne comme une pile d'assiettes :

- on ajoute toujours un élément **sur le dessus** (on dit « empiler ») ;
- on retire toujours l'élément **du dessus**, c'est-à-dire le dernier ajouté (on dit « dépiler »).

après empiler(1), empiler(2), empiler(3)	après depiler() (qui renvoie 3)
<pre> 3 <- dessus 2 1 +---+</pre>	<pre> 2 <- dessus 1 +---+</pre>

En Python, on construit une pile avec une **liste** : le dessus de la pile est la **fin** de la liste. Empiler = `liste.append(x)` ; dépiler = `liste.pop()`.

Questions

1. Une **file** fonctionne comme une file d'attente à un guichet : on ajoute à la fin, mais on retire le **premier arrivé**. Quelle est la différence entre une pile et une file ? Donnez un exemple de la vie courante pour chacune.
2. Pourquoi choisir la **fin** de la liste comme dessus de la pile ? Comparez `liste.pop()` (retire le dernier élément) et `liste.pop(0)` (retire le premier) : lequel est rapide, lequel est lent, et pourquoi ?
3. Dans votre classe `Pile`, pourquoi faut-il créer la liste dans `__init__` et non directement au niveau de la classe ?
4. Quelle est la complexité de `empiler` et de `depiler` ?

Exercices

B.1 — Écrire la classe `Pile`. Elle stocke ses éléments dans une liste et propose quatre méthodes :

Méthode	Ce qu'elle fait
<code>empiler(x)</code>	ajoute <code>x</code> sur le dessus
<code>depiler()</code>	retire l'élément du dessus et le renvoie ; lève une <code>IndexError</code> si la pile est vide
<code>sommet()</code>	renvoie l'élément du dessus sans le retirer
<code>est_vide()</code>	renvoie <code>True</code> si la pile ne contient aucun élément

```
p = Pile()
p.empiler(1)
p.empiler(2)
p.sommet()      # 2    (la pile contient toujours 1 et 2)
p.depiler()     # 2    (la pile ne contient plus que 1)
p.depiler()     # 1
p.est_vide()    # True
p.depiler()     # IndexError : la pile est vide
```

B.2 — Parenthèses équilibrées. Un texte est **équilibré** si chaque caractère ouvrant ([{ est refermé par le bon caractère fermant)] }, et dans le bon ordre : un caractère fermant doit toujours correspondre au **dernier** caractère ouvrant encore non refermé. En utilisant votre classe `Pile`, écrire `parentheses_equilibrees(texte)` qui renvoie `True` ou `False`.

```
parentheses_equilibrees("([]{})") # True
parentheses_equilibrees("([)]")  # False : "]" alors que "[" est ouvert
parentheses_equilibrees("(("))   # False : deux "(" ne sont jamais refermées
parentheses_equilibrees("())")   # False : la dernière ")" ne ferme rien
```

Exemple pas à pas pour "([)]" : on lit (→ on l'empile ; on lit [→ on l'empile ; on lit) → le dessus de la pile est [, qui ne correspond pas à) → le texte n'est pas équilibré.

Partie C — numpy et matplotlib

Questions

1. Pourquoi `numpy` est-il plus rapide qu'une boucle Python ?
2. Pourquoi comparer deux nombres décimaux avec `numpy.isclose` plutôt qu'avec `==` ?

Exercices

C.1 — Boucle contre `numpy`. Calculer la somme des carrés d'un million de nombres aléatoires, d'abord avec une boucle Python, puis avec `numpy`. Mesurer les deux temps (`time.perf_counter`) et vérifier que les résultats sont égaux, avec `==` puis avec `numpy.isclose`, et expliquer la différence.

C.2 — Graphiques. À partir du CSV de vos taux (séance 1), produire une figure avec deux graphiques côte à côte : la courbe des taux, et l'histogramme des variations quotidiennes en %. Titres, axes légendés, export PNG.